

Exame de Programação Funcional

26 de Junho de 2013

DCC/FCUP

Versão A

Nome: _____

Nº mecanográfico: _____

- Este exame contém 6 questões em 4 páginas.
- Responda às questões 1, 2, 3 no espaço marcado no enunciado.
- Responda às questões 4, 5 e 6 numa folha de exame separada.
- Duração: 2h + 30m tolerância.

1. (16%) Responda a cada uma das seguintes questões, indicando **apenas** o resultado de cada expressão.

(a) `tail (reverse [1,2,3])` = _____

(b) `filter (/='a') "abba"` = _____

(c) `[(x,y) | (x,y) <- [(1,2),(2,3)], x*y==6]` = _____

(d) `head (zip [1..10] (tail [1..10]))` = _____

(e) `foldr (+) 7 [1,2,3]` = _____

(f) Indique um tipo admissível para `head [[1],[2,3],[4,5,6]]`:

(g) Indique um tipo admissível para `map (>0)`:

(h) Considere a seguinte definição da função `last` do prelúdio-padrão

```
last [x]      = x
last (x:xs)   = last xs
```

Indique o tipo mais geral desta definição:

2. (20%) Escreva uma definição da função `maxpos :: [Int] -> Int` que calcula o máximo dos inteiros positivos numa lista; se a lista não tiver número positivos o resultado deve ser 0. Tenha a atenção de que a função esteja corretamente definida para a lista vazia e com apenas um elemento. Exemplos:

```
maxpos [1,2,3,4,5]    == 5
maxpos [-1,-2,-3,4,-5] == 4
maxpos [-1,-2,-3]     == 0
maxpos [2]            == 2
maxpos []             == 0
```

Pode utilizar funções do prelúdio-padrão e/ou listas em compreensão mas **não deve** usar recursão.

3. (20%) Escreva uma definição recursiva da função `dups :: [a] -> [a]` que duplica os valores em posições alternadas numa lista (isto é, duplica o primeiro elemento, não duplica o segundo, duplica o terceiro, e assim sucessivamente). Tenha a atenção de definir a função para a lista vazia e com apenas um elemento.

Exemplos:

```
dups "abcdef"    == "aabccdeef"  
dups [0,1,2,3,4] == [0,0,1,2,2,3,4,4]  
dups [4]         == [4,4]  
dups []          == []
```

Responda às questões 4, 5 e 6 numa folha de exame separada.

4. (15%) Considere a definição da função `drop` do prelúdio-padrão:

```
drop 0 xs          = xs          -- drop.0
drop n []          | n>0 = []      -- drop.1
drop n (x:xs)      | n>0 = drop (n-1) xs -- drop.2
```

Usando indução sobre n , prove que

$$\text{drop } m (\text{drop } n \text{ } xs) = \text{drop } (m + n) \text{ } xs$$

para todos inteiros m, n e listas xs .

5. (15%) Considere a definição em Haskell dum tipo de dados para *multi-conjuntos* (i.e. coleções sem ordem mas com repetições) representado como árvore de pesquisa:

```
data MConj a = Vazio | No a Int (MConj a) (MConj a)
```

Cada nó contém um valor e a sua multiplicidade (i.e. o número de repetições); para facilitar a pesquisa, a árvore deve estar ordenada pelos valores. Por exemplo: `No 'A' 2 Vazio (No 'B' 1 Vazio Vazio)` representa o multi-conjunto $\{A, A, B\}$ com dois caracteres 'A' e um 'B'.

(a) Escreva uma definição recursiva da função

```
ocorre :: Ord a => a -> MConj a -> Int
```

que procura o número de ocorrências de um valor num multi-conjunto; o resultado deve ser 0 se o valor não pertencer ao multi-conjunto.

(b) Escreva uma definição recursiva da função

```
inserir :: Ord a => a -> MConj a -> MConj a
```

que insere um valor num multi-conjunto mantendo a árvore de pesquisa ordenada.

6. (14%) Usando indução sobre árvores, prove a seguinte propriedade sobre as funções `ocorre` e `inserir` que definiu no exercício 5:

$$\text{ocorre } x (\text{inserir } x \text{ } t) = 1 + \text{ocorre } x \text{ } t$$

para qualquer valor x e multi-conjunto t .